

DEPARTMENT OF

COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment 3

Name : Arjun Dubey

UID 23BCS13190

Branch: BE-CSE

Section/Group: KRG-3B

Semester: 6th

Date of Performance: -02-02-2026

Subject Name: Full Stack-II

Subject Code: 23CSH-309

1. Aim: The aim of this implementation is To implement centralized state management in the EcoTrack application using Redux Toolkit and to handle asynchronous data operations using Redux async thunks with proper loading and error states..

2. Objective:-

- Configured a Redux store in a React application using Redux Toolkit
- Created and integrated Redux slices for managing application data
- Implemented asynchronous operations using Redux async thunks
- Managed loading, success, and error states during API calls
- Connected React components to Redux state using React-Redux hooks

3. Implementation/Code:

The **EcoTrack application** uses **Redux Toolkit** for centralized state management, ensuring predictable data flow and easier debugging. A global Redux store is configured using `configureStore` with feature-based slices like `logsSlice`, while asynchronous operations are handled using **Redux Async Thunks** to manage API request states. **React-Redux hooks** (`useDispatch`, `useSelector`) connect components to the store, enabling data fetching and state access. **Selectors** derive filtered views from the Redux state without mutation, and loading and error states are managed to improve performance and user experience.

- /store/logSlice.js :-

```

1  import { createSlice,createAsyncThunk } from "@reduxjs/toolkit";
2  export const fetchLogs = createAsyncThunk(
3    'logs/fetchLogs',
4    async () => {
5      await new Promise(resolve => setTimeout(resolve, 1000));
6      return [
7        { id: 1,activity:"Car Travel",carbon:4},
8        { id: 2,activity:"Electricity Usage",carbon:6},
9        { id: 3,activity:"Cycling",carbon:0},
10     ];
11   }
12 );
13
14 const logsSlice = createSlice({
15   name: 'logs',
16   initialState: {
17     data: [],
18     status: 'idle',
19     error: null,
20   },
21   reducers: {},
22   extraReducers: (builder) => {
23     builder
24       .addCase(fetchLogs.pending, (state) => {
25         state.status = 'loading';
26       })
27       .addCase(fetchLogs.fulfilled, (state, action) => {
28         state.status = 'succeeded';
29         state.data = action.payload;
30       })
31       .addCase(fetchLogs.rejected, (state, action) => {
32         state.status = 'failed';
33         state.error = action.error.message;
34       });
35   },
36 });
37 export default logsSlice.reducer;
38

```

- store.js :-

```

1  import {configureStore} from '@reduxjs/toolkit';
2  import logsReducer from './logsSlice';
3  const store = configureStore({
4    reducer: {
5      logs: logsReducer,
6    },
7  });
8  export default store;

```

- **LogSlice.jsx :-**

```
• import { createSlice, createAsyncThunk } from
"@reduxjs/toolkit";
•
• export const fetchLogs =
createAsyncThunk("logs/fetchLogs", async () => {
•   return new Promise((resolve) =>
•     setTimeout(
•       () =>
•         resolve([
•           { id: 1, activity: "Commute", carbon: 2.5 },
•           { id: 2, activity: "Lunch", carbon: 0.5 },
•         ]),
•       1000
•     )
•   );
• });
•
• const logSlice = createSlice({
•   name: "logs",
•   initialState: {
•     data: [],
•     status: "idle",
•     error: null,
•   },
•   reducers: {},
•   extraReducers: (builder) => {
•     builder
•       .addCase(fetchLogs.pending, (state) => {
•         state.status = "loading";
•       })
•       .addCase(fetchLogs.fulfilled, (state, action) => {
•         state.status = "succeeded";
•         state.data = action.payload;
•       })
•       .addCase(fetchLogs.rejected, (state, action) => {
•         state.status = "failed";
•         state.error = action.error.message;
•       })
•   },
• });
```

```
•  
• export default logSlice.reducer;  
•  
•
```

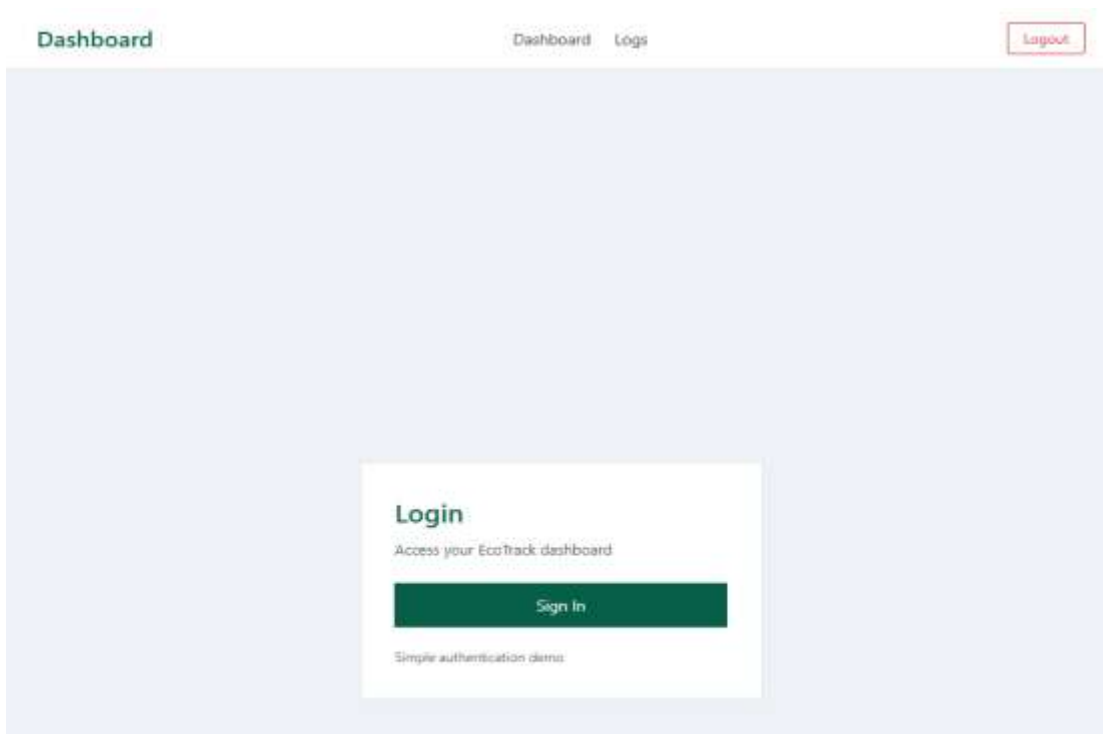
- App.jsx:-

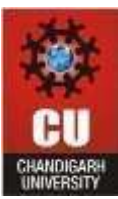
```
1 import Dashboard from "../pages/dashboard.jsx";  
2 import Log from "../pages/log.jsx";  
3 import Header from "../components/Header.jsx";  
4 import { Routes, Route, Link, BrowserRouter } from "react-router-dom";  
5 import Login from "../pages/login.jsx";  
6 import ProtectedRoute from "../Routes/ProtectedRoute.jsx";  
7 import DashboardLayout from "../pages/dashboardLayout.jsx";  
8  
9 function App() {  
10   return (  
11     <>  
12       <BrowserRouter>  
13         <Header title="Dashboard" />  
14         <main>  
15           <Routes>  
16             <Route path="/login" element={ <Login /> } />  
17             <Route  
18               path="/dashboardLayout"  
19               element={  
20                 <ProtectedRoute>  
21                   <DashboardLayout />  
22                 </ProtectedRoute>  
23               } />  
24             <Route  
25               path="/" />  
26               element={  
27                 <ProtectedRoute>  
28                   <Dashboard />  
29                 </ProtectedRoute>  
30               } />  
31             <Route  
32               path="/log"  
33               element={  
34                 <ProtectedRoute>  
35                   <Log />  
36                 </ProtectedRoute>  
37               } />  
38             </Routes>  
39           </main>  
40         </BrowserRouter>  
41       </>  
42     );  
43   }  
44  
45   export default App;  
46  
47  
48  
49
```

- ProtectRoute.jsx

```
1  import {Navigate} from "react-router-dom";
2  import {useAuth} from "../context/AuthContext";
3
4  const ProtectedRoute = ({children}) => {
5      const {isAuthenticated} = useAuth();
6
7      if(!isAuthenticated){
8          return <Navigate to="/login" replace />;
9      }
10     return children;
11 }
12 export default ProtectedRoute;
```

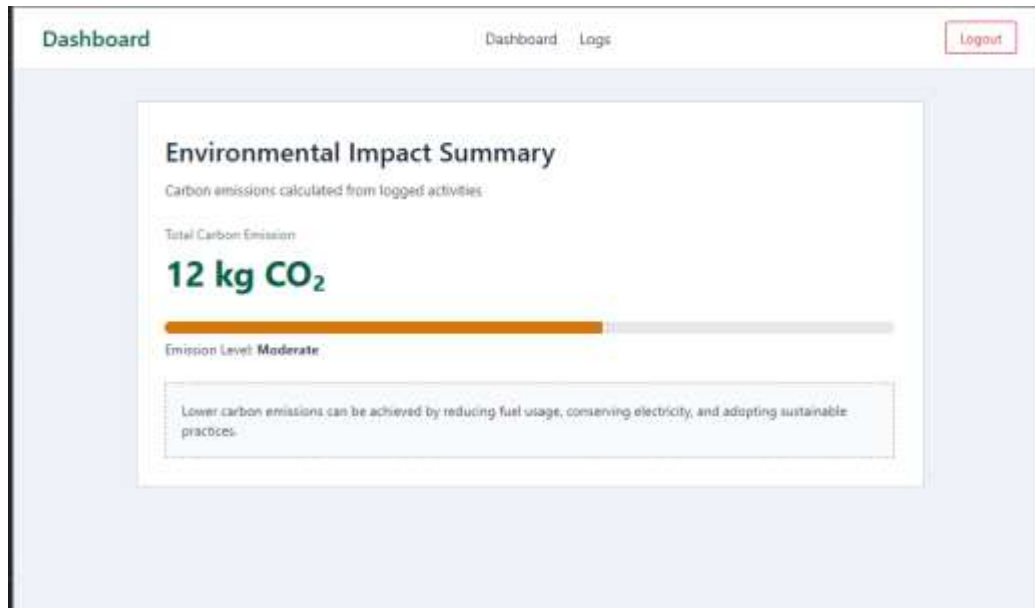
4. Output





DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.



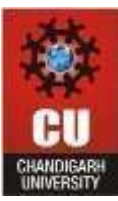
Dashboard [Dashboard](#) [Logs](#) [Logout](#)

Activity Logs

Track your activities and their carbon impact

Activity	Carbon Emission (kg CO ₂)
Car Travel	4
Electricity Usage	6
Public Transport	2

Tip: Choose sustainable options like biking, walking, or public transport to lower your emissions.



DEPARTMENT OF

COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

5. Learning Outcome :-

- Learned how to configure a Redux store and create feature-based slices
- Understood the use of async thunks for managing asynchronous operations
- Gained experience in handling loading, success, and error states in Redux
- Learned to derive filtered views using selectors without mutating the global state